

# EvoPIE Manual

Alessio Gaspar, Paul Wiegand, Dmytro Vitel

August 13, 2026

## Contents

|          |                                          |          |
|----------|------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                      | <b>2</b> |
| <b>2</b> | <b>DB Schema</b>                         | <b>3</b> |
| <b>3</b> | <b>Switching Quiz Models</b>             | <b>4</b> |
| <b>4</b> | <b>SSL certificates</b>                  | <b>5</b> |
| <b>5</b> | <b>Sampling justifications algorithm</b> | <b>5</b> |
| <b>6</b> | <b>Grading Formula</b>                   | <b>6</b> |
| <b>7</b> | <b>Extensibility &amp; Debugging</b>     | <b>7</b> |

# 1 Introduction

**EvoPIE** is Evolutionary Peer Instruction Environment. It operationalizes peer instruction, an active learning method in which students first answer questions individually, then may see peers' judgments of incorrect alternatives, and revise their answers.

The implemented procedure is **asynchronous**, meaning that interactions between the instructor and the student, and between students, are decoupled in time.

The implemented approach is also **data-driven**. The adjusted set of exercises depends on interaction outcomes. The stateful background evolutionary process maintains the model, which we dub the **quiz model**, a probability distribution on possible questions and distractors. The goal of this process is to adjust this distribution based on interaction outcomes to uncover **underlying student misconceptions**, which may then be used by the instructor to improve the student's knowledge.

To summarize, the system's features include the following.

**MCQ knowledge probing software.** Instructors develop sets of quizzes and questions, and maintain student groups, controlling the time of taking the exercise. Student attempts are registered and reported back to the instructor, demonstrating the student's performance on different procedures **steps**.

1. **Step 1** - student takes initial quiz, providing justifications for each unselected option, explaining reasons why it may seem incorrect.
2. **Step 2** happens on a set deadline or when the instructor manually moves the quiz to this step. Students can now see their peers' previous selections and justifications, which provide hints about incorrect options. Each student should provide the specified number of justifications to get a full grade.
3. **Step 3 (Optional)** - student may provide their own distractors, which undergo moderation and approval by instructor to be added to the pool of alternative options as possible good representatives of misconceptions.
4. **Solutions** step allows students to see the correct answers to the given quizzes.

EvoPIE provides a web interface for communicating with stored exercises, both for instructors and students. As a web application is stateless, all session-level and quiz-level information is serialized to the database. Important stateful procedures and algorithms:

1. Sampling of the quiz model for a new quiz (Step 1). The quiz is cached in the student's attempt record in the DB and returned on each page refresh.
2. Sampling of justification for peer instruction (entering Step 2). The algorithm considers only justifications that are already in the DB. Sampling attempts to give equal chances to the justifications of different students. Justifications are saved in the DB and returned on page refresh on Step 2.
3. Quiz model state (Step 1). The quiz model has a simple interface for generating quizzes and providing interaction outcomes for adjustments. The state persists across sessions and multiple students, so it requires some synchronization for concurrent update attempts. This is handled with an optimistic model of parallel transactions, implemented in SQLAlchemy ORM, which would fail and request a retry of the operation when two transactional modifications run concurrently.

This guide provides further technical details on implementation and is partially based on materials already published in the GitHub README and the EvoPIE project's CEREAL drive documentation.

## 2 DB Schema

Figure 1 presents the current entities in the schema, and Table 1 explains their meaning.

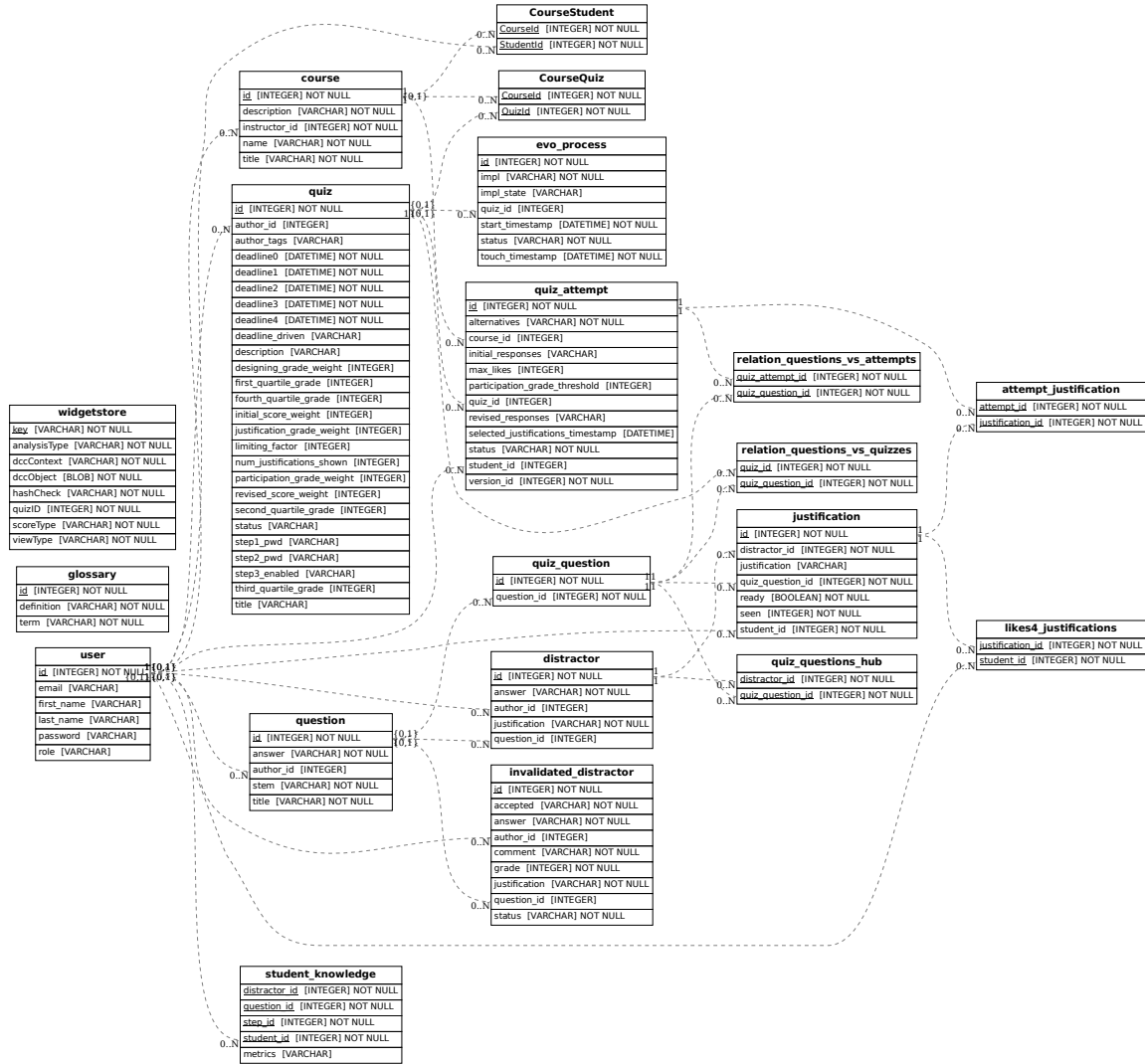


Figure 1: Current DB Schema and relations between entities

**Notes for practitioners.** The `sqlite3` command line tool becomes handy when you need to investigate present entities in the concrete database checkpoint (`.sqlite` file). You may use it on `evopie` server, but better to create the DB backup first.

```
sqlite3 DB_quizlib.sqlite
```

Useful commands in interactive mode:

- `.mode line` - outputs formatted entities from queries (property name = value), while `.mode list` outputs in compact format without property name annotations;
- `.schema [table]` outputs the table(s) structure as DDL statements, to investigate types of present entity properties;
- `.table` - lists all present tables;
- `.help` provides full set of meta commands;

- interactive mode allows execution DML and DDL statements, you may query entities using SQL language and perform data updates, also the tool allows data migrations (examples aer provided in db folder of github repository root).

Table 1: Database Entity Summary

| Table Name                                  | Description                                               |
|---------------------------------------------|-----------------------------------------------------------|
| <code>user</code>                           | Students, Instructors, Admins                             |
| <code>course</code>                         | Basic course info and author                              |
| <code>CourseQuiz</code>                     | Attached course quizzes (many-to-many)                    |
| <code>CourseStudent</code>                  | Attached course students (many-to-many)                   |
| <code>quiz</code>                           | Quiz basic info and attached config                       |
| <code>question</code>                       | Question-answer pool                                      |
| <code>distractor</code>                     | Question wrong options with final justification           |
| <code>quiz_question</code>                  | Question id per usage in some quiz                        |
| <code>relation_questions_vs_quizzes</code>  | Quiz to question usage id (many-to-many, for some reason) |
| <code>quiz_questions_hub</code>             | Attached distractors for question usage (distractor pool) |
| <code>quiz_attempt</code>                   | Student answers to the quiz questions (Step1,2)           |
| <code>justification</code>                  | Provided justification by student for distractor          |
| <code>attempt_justification</code>          | Attempt justifications (many to many, for some reason)    |
| <code>likes4_justifications</code>          | Justification likes from students (many-to-many)          |
| <code>relation_questions_vs_attempts</code> | Attempt question (usage)                                  |
| <code>evo_process</code>                    | Quiz model state                                          |
| <code>glossary</code>                       | Interaction postprocessing (Dr. Paul Wiegand)             |
| <code>invalidated_distractor</code>         | Proposed by students distractors (Step 3)                 |
| <code>student_knowledge</code>              | Interaction modeling, only for student simulation         |
| <code>widgetstore</code>                    | Interaction postprocessing (Dr. Paul Wiegand)             |

**Notes on schema.** Some schema decisions were not optimal and may cause confusion. Note that courses, quizzes, and questions are independent entities that are “attached” to one another via intermediate tables (usually with many-to-many relationships). Question usage (a.k.a. `quiz_question`) connects a quiz to a question, and in the UI also connects a subset of question distractors via the `quiz_question_hub` table.

### 3 Switching Quiz Models

Quiz models represent the algorithms that form quizzes for students based on previous observed interactions stored in Quiz Model state (DB table `evo_process`). Currently, we support the following samplings:

1. **Manual selection** (default, `id=None`) - relies on the instructor’s decision of what distractors should be added to the question. Basically, it uses the entire distractor pool assigned to the quiz question. Returns the same quiz each time.
2. **Random quiz model** (`id=RandomQuizModel`) - picks random 3 distractors per question from the assigned pool of distractors once at start, and returns the same quiz on each request.
3. **PPHC quiz model** (`id=PphcQuizModel`) - implements the Parallel Pareto Hill Climbing algorithm, expects a set of interactions from several students to adjust distractor selection. Survivors within parent-child pairs are determined by the Paerto domination relation. Consider `pph_quiz_model.py` for settings of the PPHC process.

4. **Sampling Strategies** (`id=SamplingQuizModel`) - implements sampling of given distractor pools based on specified computed interaction features (domination, nondomination, etc), given in configuration of the process. See `sampling_quiz_modle.py`.

**To switch between different quiz models**, consider helper functions in `quiz_model.py` file (which also contains the base interface to implement for new Quiz Generation Algorithms). The function `set_quiz_model` is used in `evopie/__init__.py` to set the necessary algorithm class and settings (classes are provided in the list above as ids). Function `get_quiz_builder` creates a quiz model builder which restores the algorithm state from the DB table `evo_process` in a new instance of `QuizModel`. The general stateless schema of a web request calls `load_quiz_model`, then either samples a new quiz (`quiz_model.get_for_evaluation(current_user.id)`), or updates the algorithm state with student answers (`quiz_model.evaluate(sid, answers)`), and then saves the state back to the DB.

## 4 SSL certificates

Instruction is copied from CEREAL/DEV & Data Collection/Letsencrypt.

Let's Encrypt certificates are issued on the Evopie server with `certbot` command. The certificate renewal cycle is 90 days, and it currently works automatically through the following hooks. To see live certificates, you may execute `certbot certificates`.

Location `/etc/letsencrypt` contains the following folders:

- `./live` - symbolic links to currently active certificates, updated every 90 days;
- `./renewal-hooks` - scripts to propagate the certificates to evopie/evoparsons services:
  - `./pre` scripts execute before renewal:
    - `0-stop-port-80.sh` stops java-public container, ports 80/443 are used by Let's Encrypt Certbot to verify that the certificate domain belongs to us, downtime 1-2 mins;
  - `./post` scripts execute after renewal:
    - `0-pkcs-convert.sh` converts pem certs to pkcs12 p12 file for evoparsons containers
    - `1-port-start-80.sh` starts java-public
    - `2-container-restart.sh` restarts all containers that cached previous certificates (for evopie it is `evopie_nginx_1` only, for evoparsons - all jetty containers)

**Each time when you add new container which uses and caches certs**, you have to add corresponding restart line to `2-container-restart.sh`. All scripts are placed on Google Drive.

## 5 Sampling justifications algorithm

The system distinguishes quiz steps (Step 1, 2, 3) as stages of the Quiz entity scheduled by the instructor and as stages of the student quiz attempt entity, indicating the stage at which the current student is in the overall flow. A quiz attempt is created when the student enters Step 1 (the student clicks the "Take Quiz" button and optionally authenticates themselves if the quiz is protected with a passphrase). On Step 1, the student provides justifications, the reasons why the proposed alternative options are incorrect. These are saved to the DB for Step 2 peer instruction. Entering Step 2 then causes the available justifications to be sampled to guide the student and to collect the student's evaluation of the usefulness of the justifications.

This selection happens in `routes_pages.py`, `select_justifications` routine, where we apply the sampling policy:

```

selection_policy = j_slot_group_till(num_in_group,\
    in_group_policy=j_least_seen(j_random), \
    out_group_policy=\
        j_tournament(get_k_tournament_size, fitness=
            get_justification_fitness))

```

This policy is formed from combinators (check different `j_*` functions in the same file) that describe the sampling. The current one splits the slots onto two subgroups (60%-40%, hardcoded in `config.py`). First subgroup picks the least seen justifications from the pool of whole justifications, splitting ties at random (`j_least_seen(j_random)`). The second subgroup is selected using a tournament algorithm based on fitness, i.e., the student's Step 1 score (assuming that better-performing students provide better justifications). Again, this strategy may be further adjusted with the provided `j_*` functions-combinators.

After sampling, justifications are cached in the database in the `attempt_justification` relation and are always returned from the DB on a new session or refresh. Consider [this GitHub Wiki Page](#) for additional details.

## 6 Grading Formula

The final grade is determined by student performance and participation in each quiz step. The UI page that provides this information is `/quiz/<int:qid>/<int:course_id>/grades`. It allows adjusting the weights  $w_i$  per step. Routine `get_quiz_statistics` computes the grades. Grade components:

1.  $s_0$  - step 1 score, MCQ % of correct answers on Step 1.
2.  $s_1$  - step 2 score, a.k.a revised score, the student may adjust the initial answers based on the provided peer justifications.
3.  $s_2$  - justification score, based on given likes by other students.
4.  $s_3$  - participation grade, based on likes provided in the expected range of likes.
5.  $s_4$  - designing grade, only counts if Step 3 is enabled, on which the student provides the alternative distractor, which is then manually evaluated by the instructor.

Computation of justification and participation grades follows the instructions provided on [this Github Wiki Page](#) as follows.

**Computing the participation grade  $s_3$ .** For a given quiz with  $Q$  questions, with  $A_q$  alternatives per question  $q$ , we have a total of  $A = \cup_{q \in Q} A_q$  alternatives available to the student. Each of these alternatives  $a \in A$  will be shown, during Step 2, along with a variable number of justifications  $J_a$ , with the total number of such justifications being  $J = \cup_{a \in A} J_a$ . The Limiting Factor  $l \in [0, 1]$  is a parameter set by the instructor as a percentage representing the number of likes desired from each student to indicate that they have participated fully. The maximum number of likes available to give is  $|J|$ . Participation Threshold is  $l * |J|$ . If a student provides a number of likes in the range  $[\text{round}(l * |J| * 0.8), l * |J|]$ , they get full participation credit  $s_3 = 1$ , 0 otherwise.

**Computing the justifications grade  $s_2$ .** First, we compute the score for each student  $s \in S$ , in a given quiz:

$$\text{score} = \sum_{k \in S, k \neq s} \text{Likes}(k, s) * \min\left(\frac{l * |J|}{\text{Likes}(k)}, 1\right)$$

- where  $\text{Likes}(k, s)$  is the number of likes given by  $k$  to  $s$  (in this quiz);  $\text{Likes}(k)$  is the total number of likes given by student  $k$  (in this quiz). The grade  $s_2$  is then assigned based on the quartile in which the student's score places them, compared to their peers:

- $[0 : 25\%] \rightarrow 1$  point ( $s_2 = 0.25$ );
- $(25\% : 50\%] \rightarrow 2$  points ( $s_2 = 0.5$ );
- $(50\% : 75\%] \rightarrow 3$  point ( $s_2 = 0.75$ );
- $(75\% : 100\%] \rightarrow 4$  point ( $s_2 = 1$ ).

To summarize, the `get_quiz_statistics` computes maps of the given and received likes, and then computes like scores (including those mentioned in the Wiki normalization of scores based on evaluator participation) across all students. Then, from the ordered scores, the routine computes the quartiles and determines where each student's performance falls. Based on the quartile, we assign a number of points (part of the Quiz Configuration in the Instructor UI), then normalize it to 100%. This forms the final  $s_2$  score.

## 7 Extensibility & Debugging

Installation steps are provided in the GitHub README and on [this Wiki page](#). Here, we elaborate more on how the system may be extended with new CLI **entry points** and configure a debug session.

We use `pipenv` for environment management, activation, installing new packages, and executing entry points. CLI entry points are organized in `AppGroup` (see `cli.py`). CLI entry points include routines that run student simulation by controlling the web context and executing Blueprint endpoints directly. The `slurm` folder contains examples of shell scripts for running CLI endpoints and experiments with simulated student groups. For instance:

```
export EVOPIE_DATABASE_URI=sqlite:/// $WORK/evopie/data-$SLURM_ARRAY_TASK_ID/db.sqlite
# export EVOPIE_DATABASE_LOG=$WORK/evopie/data-$SLURM_ARRAY_TASK_ID/db.log
export PYTHONPATH=$(pipenv run which python)
pipenv run flask quiz deca-experiments \
  --deca-spaces $WORK/evopie/data-$SLURM_ARRAY_TASK_ID/deca-spaces \
  --algo-folder $WORK/evopie/data-$SLURM_ARRAY_TASK_ID/algo \
  --results-folder $WORK/evopie/data-$SLURM_ARRAY_TASK_ID/results \
  --random-seed 23 --num-runs 30 \
  --algo '{ "id": "pphc-1-2-1-3", "algo": "evopie.pphc_quiz_model.PphcQuizModel", "pop_size": 1, "pareto_n": 2, "child_n": 1, "gene_size": 3 }'
```

This example demonstrates how evolutionary processes may be simulated with different assumed student knowledge structures and different algorithms for uncovering the knowledge.

**Notes to practitioners.** CLI commands may be of a different nature. For instance, you may want to add endpoints to visualize crude data from the DB as matplotlib PDF figures or LaTeX tables. To debug such endpoints, consider modifying `.vscode/launch.json` to create a new debug configuration that provides a standard debugging experience with breakpoints and step-by-step execution. Examples:

```
{
  "name": "CLI: DB inspect",
  "type": "python",
  "request": "launch",
  "module": "flask",
  "env": {
    "FLASK_APP": "app.py",
```

```
    "FLASK_ENV": "development",
    "FLASK_DEBUG": "1"
  },
  "args": ["DB-inspect"]
},
{
  "name": "CLI: quiz init",
  "type": "python",
  "request": "launch",
  "module": "flask",
  "env": {
    "FLASK_APP": "app.py",
    "FLASK_ENV": "development",
    "FLASK_DEBUG": "1"
  },
  "args": ["quiz", "init", "-nq", "3", "-nd", "5"]
},
```